

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Laboratorio 5: SQL

Sistemas de Información para la Gestión

Entrega individual · ICN-292 · Paralelo 100

Joaquín Tapia Mandiola
Ingeniería Civil Industrial

Repositorio GitHub

https://github.com/JotaReyyyyyyyyyyy/ICN292-Lab5-Tapia-Joaquin

Noviembre, 2026

Campus Vitacura

Consulta 1

```
SELECT v.nombre, COUNT(ve.id_venta) AS cantidad_ventas
FROM vendedores AS v
LEFT JOIN ventas AS ve ON ve.id_vendedor = v.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY cantidad_ventas DESC, v.nombre;
```

Result Grid | Filter Rows:

nombre	cantidad_ventas
Camila Ortiz	4
Diego Fuentes	3
Ignacio Vera	3
Javiera Soto	3
Paula Riquelme	3
Matías Herrera	2
Fernanda Lillo	0
Rubén Castro	0

Consulta 2

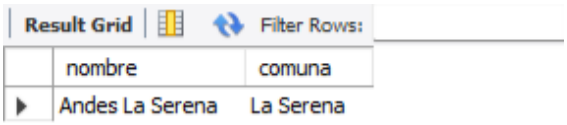
```
SELECT t.nombre AS tienda,
       COUNT(ve.id_venta) AS cantidad_ventas,
       COALESCE(SUM(ve.unidades), 0) AS unidades_vendidas
FROM tiendas AS t
LEFT JOIN vendedores AS v ON v.id_tienda = t.id_tienda
LEFT JOIN ventas AS ve ON ve.id_vendedor = v.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades_vendidas DESC, t.nombre;
```

Result Grid | Filter Rows: | Export:

tienda	cantidad_ventas	unidades_vendidas
Andes Concepción	6	14
Andes Providencia	7	11
Andes Maipú	5	10
Andes La Serena	0	0

Consulta 3

```
SELECT t.nombre, t.comuna
FROM tiendas AS t
LEFT JOIN vendedores AS v ON v.id_tienda = t.id_tienda
WHERE v.id_vendedor IS NULL;
```



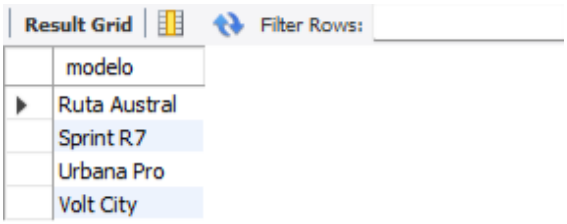
Result Grid | Filter Rows:

	nombre	comuna
▶	Andes La Serena	La Serena

Consulta 4

```
SELECT b.modelo
FROM bicicletas AS b
LEFT JOIN ventas AS ve
       ON ve.id_bicicleta = b.id_bicicleta
      AND ve.fecha >= '2025-06-01' AND ve.fecha < '2025-07-01'
WHERE ve.id_venta IS NULL
ORDER BY b.modelo;
```

La condición del mes va en el `ON` del `LEFT JOIN`; si la hubiera puesto en el `WHERE`, la consulta no devolvería filas, porque los modelos sin venta en junio llegan con la fecha en `NULL` y no cumplen el filtro.



Result Grid | Filter Rows:

	modelo
▶	Ruta Austral
	Sprint R7
	Urbana Pro
	Volt City

Consulta 5

```
SELECT v.nombre, t.nombre AS tienda
FROM vendedores AS v
LEFT JOIN tiendas AS t ON t.id_tienda = v.id_tienda
LEFT JOIN ventas AS ve ON ve.id_vendedor = v.id_vendedor
WHERE ve.id_venta IS NULL
ORDER BY v.nombre;
```

Result Grid | Filter Rows:

nombre	tienda
Fernanda Lillo	Andes Providencia
Rubén Castro	NULL

Consulta 6

```
SELECT a.nombre AS vendedor_antiguo, a.fecha_ingreso AS ingreso_antiguo,
       b.nombre AS vendedor_nuevo, b.fecha_ingreso AS ingreso_nuevo,
       t.nombre AS tienda
FROM vendedores AS a
JOIN vendedores AS b
  ON b.id_tienda = a.id_tienda
 AND (a.fecha_ingreso < b.fecha_ingreso
      OR (a.fecha_ingreso = b.fecha_ingreso AND a.id_vendedor < b.id_vendedor))
JOIN tiendas AS t ON t.id_tienda = a.id_tienda
ORDER BY t.id_tienda, a.fecha_ingreso, b.fecha_ingreso;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

vendedor_antiguo	ingreso_antiguo	vendedor_nuevo	ingreso_nuevo	tienda
Diego Fuentes	2024-01-15	Camila Ortiz	2024-06-20	Andes Providencia
Diego Fuentes	2024-01-15	Fernanda Lillo	2025-02-03	Andes Providencia
Camila Ortiz	2024-06-20	Fernanda Lillo	2025-02-03	Andes Providencia
Javiera Soto	2023-07-24	Matías Herrera	2024-01-15	Andes Maipú
Paula Riquelme	2022-11-02	Ignacio Vera	2024-08-19	Andes Concepción

Consulta 7

```
SELECT t.nombre AS tienda, SUM(ve.unidades * b.precio) AS monto_montana_ruta
FROM tiendas AS t
JOIN vendedores AS v ON v.id_tienda = t.id_tienda
JOIN ventas AS ve ON ve.id_vendedor = v.id_vendedor
JOIN bicicletas AS b ON b.id_bicicleta = ve.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(ve.unidades * b.precio) > 2000000
ORDER BY monto_montana_ruta DESC;
```

Result Grid | Filter Rows:

tienda	monto_montana_ruta
Andes Providencia	4399930.00
Andes Concepción	3499950.00

Consulta 8

```
SELECT modelo, precio
FROM bicicletas
WHERE precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY precio DESC;
```

Result Grid | Filter Rows:

modelo	precio
Volt City	1249990.00
Sprint R7	1099990.00
Ruta Austral	899990.00
Trail Master	749990.00

```
SELECT AVG(precio) AS promedio_catalogo
FROM bicicletas;
```

Result Grid | Filter Rows:

promedio_catalogo
657490.000000

El promedio del catálogo es 657.490,00. Lo calcula la subconsulta `SELECT AVG(precio) FROM bicicletas` del `WHERE`; la segunda sentencia lo muestra por separado.

Consulta 9

```
SELECT v.nombre, AVG(ve.unidades) AS promedio_unidades
FROM vendedores AS v
JOIN ventas AS ve ON ve.id_vendedor = v.id_vendedor
GROUP BY v.id_vendedor, v.nombre
HAVING AVG(ve.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY promedio_unidades DESC, v.nombre;
```

Result Grid | Filter Rows:

nombre	promedio_unidades
Paula Riquelme	2.6667
Javiera Soto	2.3333
Diego Fuentes	2.0000
Ignacio Vera	2.0000

Consulta 10

```
SELECT v.nombre, SUM(ve.unidades * b.precio) AS monto_total
FROM vendedores AS v
LEFT JOIN ventas AS ve ON ve.id_vendedor = v.id_vendedor
LEFT JOIN bicicletas AS b ON b.id_bicicleta = ve.id_bicicleta
GROUP BY v.id_vendedor, v.nombre
ORDER BY monto_total DESC;
```

Fernanda Lillo y Rubén Castro muestran `NULL` porque no tienen ventas: el `LEFT JOIN` los deja sin filas que sumar y `SUM` devuelve `NULL`. Para que mostraran cero habría que usar `COALESCE(SUM(ve.unidades * b.precio), 0)`.

Result Grid | Filter Rows:

nombre	monto_total
Camila Ortiz	3429950.00
Paula Riquelme	3149920.00
Matías Herrera	3049970.00
Diego Fuentes	3009940.00
Ignacio Vera	2319940.00
Javiera Soto	2009930.00
Fernanda Lillo	NULL
Rubén Castro	NULL